LR4 — Managed Container Web Application Deployment

Topic

Deployment and launch of a containerized web application using Azure App Service and Docker.

Objective

Deploy the same containerized Workout Planner web application as a managed service using Azure App Service, and compare with the VM-based deployment from LR2.

Steps Performed

1. Azure Container Registry

The same ACR (`acrworkoutplanner`) from LR2 is reused. The Docker image is already pushed.

Screenshot: *N/A (ACR was covered in LR2)*

2. Terraform Infrastructure (`infra/app-service/`)

App Service Plan

- `azurerm_service_plan` — Linux, SKU B1 (minimum paid tier)
- Supports container deployments natively

Linux Web App

- `azurerm_linux_web_app` — pulls container from ACR
- Managed identity for secure ACR access
- HTTPS enforced (`https_only = true`)
- Environment variables configured for ML API connection

3. Deployment

```
cd infra/app-service && terraform init && terraform apply
# Or update image:
./scripts/deploy-app-service.sh
```

4. App Service Configuration

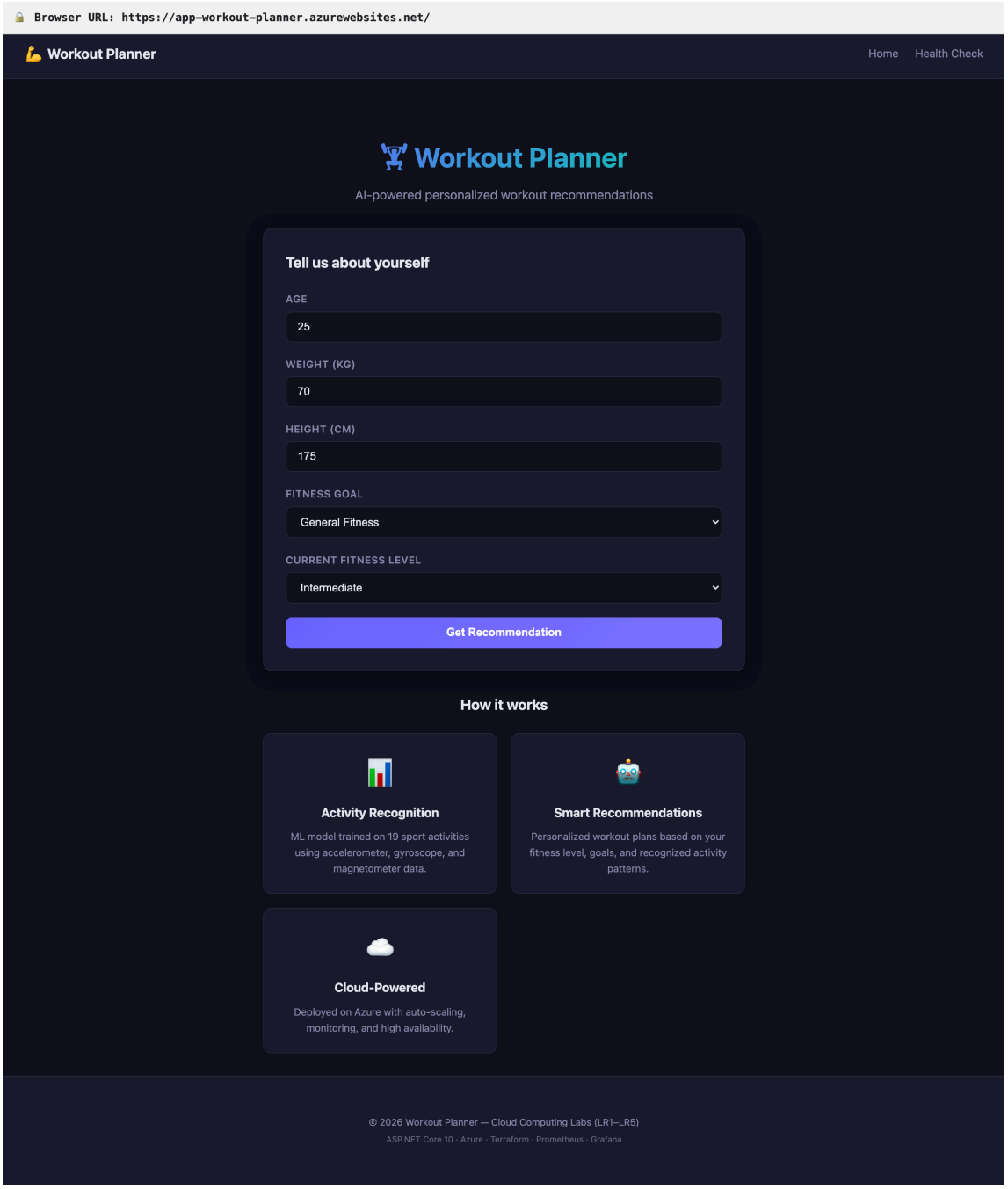
- Container source: Azure Container Registry
- Image: `workoutplanner:latest`
- HTTPS: Managed SSL certificate (automatic)
- Always On: Enabled (B1 plan)

Screenshot: *App Service configured via Terraform*

5. Running Application

- **URL**: `https://app-workout-planner.azurewebsites.net`

Screenshots:



Browser URL: https://app-workout-planner.azurewebsites.net/

Workout Planner

Home Health Check

Workout Planner

AI-powered personalized workout recommendations

Tell us about yourself

AGE

30

WEIGHT (KG)

80

HEIGHT (CM)

180

FITNESS GOAL

Muscle Gain

CURRENT FITNESS LEVEL

Intermediate

Get Recommendation

How it works

Activity Recognition

ML model trained on 19 sport activities using accelerometer, gyroscope, and magnetometer data.

Smart Recommendations

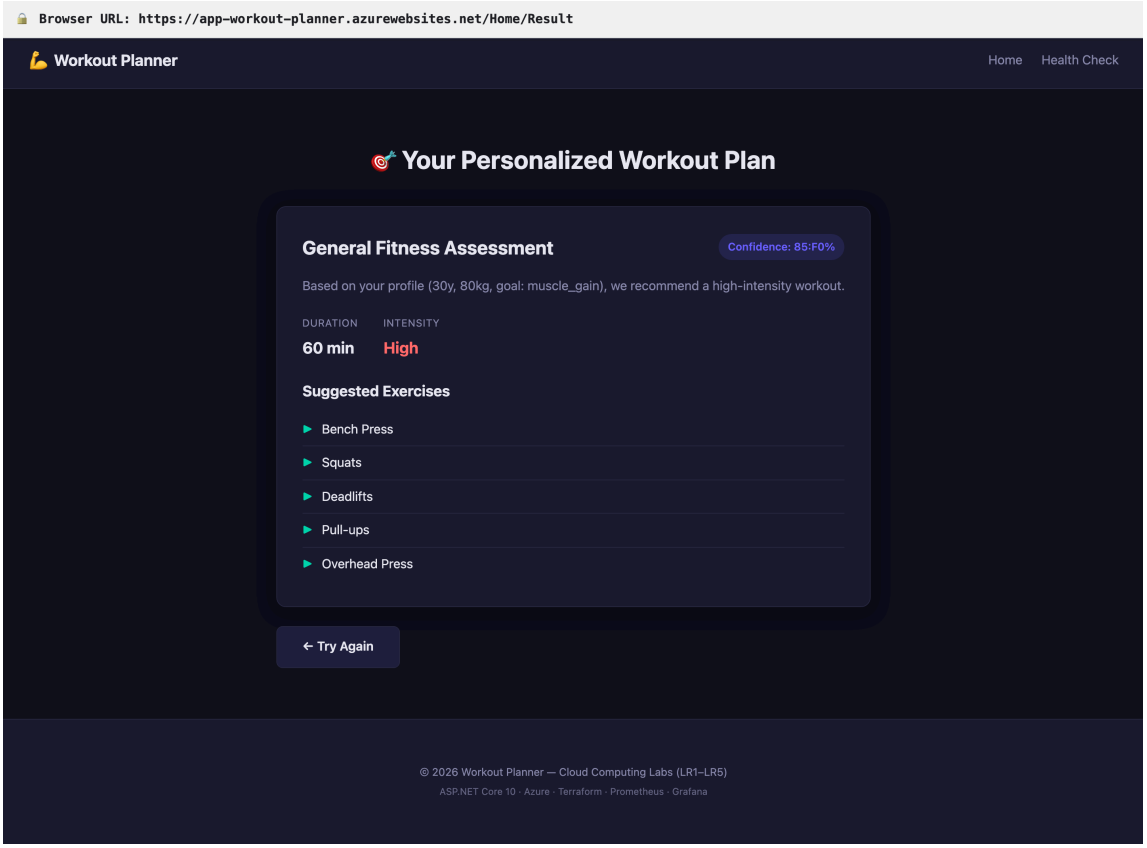
Personalized workout plans based on your fitness level, goals, and recognized activity patterns.

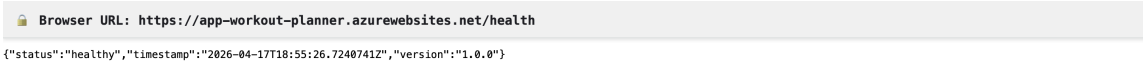
Cloud-Powered

Deployed on Azure with auto-scaling, monitoring, and high availability.

© 2026 Workout Planner — Cloud Computing Labs (LR1–LR5)

ASP.NET Core 10 · Azure · Terraform · Prometheus · Grafana





{"status":"healthy","timestamp":"2026-04-17T18:55:26.7240741Z","version":"1.0.0"}

LR2 vs LR4 Comparison

Aspect	LR2 — VM Deployment	LR4 — App Service
Compute Model	IaaS (Infrastructure as a Service)	PaaS (Platform as a Service)
Control Level	Full (SSH, root access, OS management)	Limited (managed by Azure)
Setup Complexity	High (cloud-init, Docker install, NSG)	Low (point to ACR image)
Scaling	Manual (resize VM or add more)	Built-in auto-scale rules
SSL/TLS	Manual (certbot, Let's Encrypt)	Automatic (Azure managed cert)
OS Updates	Your responsibility	Azure handles patching
Cost Model	Pay per VM uptime	Pay per App Service Plan
Availability SLA	Depends on VM config	99.95% built-in
Deployment Speed	Slower (VM boot + Docker pull)	Faster (container swap)
Monitoring	Must set up manually	Built-in App Service monitoring
Best For	Full control, custom configurations	Standard web apps, rapid deployment

---

Key Takeaways

1. **App Service simplifies deployment**: No need to manage OS, Docker runtime, or networking rules.
2. **HTTPS is automatic**: Azure provides managed SSL certificates at no extra cost.
3. **Scaling is built-in**: Can set auto-scale rules based on CPU, memory, or HTTP queue length.
4. **Trade-off**: Less control over the runtime environment in exchange for operational simplicity.

Results

- Same Docker image from LR2 deployed to Azure App Service
- HTTPS automatically provisioned
- Application accessible via managed domain name
- Terraform manages the entire App Service infrastructure

Conclusion

Azure App Service provides a significantly simpler deployment model compared to VM-based hosting. The same Docker image runs identically on both platforms, demonstrating container portability. For production web applications that don't require OS-level customization, App Service (PaaS) is the recommended approach due to its operational simplicity, built-in scaling, and managed SSL. The VM approach (IaaS) remains valuable for workloads requiring full control or custom runtime environments.